

README

Part 14: 推理部署实战 — vLLM

- > Part 8 06 章手写了 PagedAttention 模拟、量化与投机解码的原理；本部分把它们放上
- > 工业标准引擎 vLLM，做一次真正的 serving 实验：同一模型、同一批 prompt,
- > naive 循环 vs vLLM 的 TTFT/TPOT/吞吐对比——"手写 vs 工具"的收官之战。
- > 主源：[vllm-project/vllm](https://github.com/vllm-project/vllm) (90.5k, Apache-2.0)

学习目标

完成本部分后，你将能够：

- 理解 推理部署在 LLM 链路中的位置和价值
- 手写 朴素推理基线并测量 TTFT/TPOT/吞吐，完成与 vLLM 的公平对比
- 解释 vLLM 的核心优化（连续批处理、PagedAttention、投机解码）
- 配置 vLLM 的推理服务并理解每个参数的含义
- 识别 推理部署中的常见陷阱并设计防范策略

章节导航

序号	章节	内容	对应脚本
01	[朴素基线与对比设计](01_naive_baseline.md)	手写侧基线：TTFT/TPOT/吞吐的测量方法与指标定义	'01'
02	[vLLM 实战与对比](02_vllm_serving.md)	两案安装 → 离线推理 → OpenAI 服务 → benchmark → 量化/投机解码	—（CLI 实操）

前置知识

- 必须掌握：
- [Part 8 06 章 推理与服务](../Part8_post_training/tutorial/06_inference_and_serving.md)：memory-bound、PagedAttention、投机解码（手写模拟）——本部分的"手写侧"，对比表的另一端
 - [Part 9 GPU 执行模型](../Part9_cuda_kernels/tutorial/README.md)：异步执行/计时同步——看懂 01 章测量陷阱的底层原因
- 建议掌握：
- [Part 7 Transformer 架构](../Part7_minimind/tutorial/README.md)：KV cache 与 GQA——算 KV 显存账时直接用
- 可选：
- [Part 10 分布式](../Part10_distributed/tutorial/README.md)：多卡张量并行推理（vLLM `--tensor-parallel-size`）时用到

在 LLM 链路中的位置

预训练 → 微调/对齐(Part 8/11/12) → 【本部分: 推理与服务（模型变产品）】
↑
你在这里

为什么推理部署是"模型变产品"的关键：

证据	说明
成本	推理成本占 LLM 总成本的 70%+
速度	用户体验取决于首 token 延迟（TTFT）和生成速度（TPOT）
规模	需要处理并发请求，而非单个请求

理论背景

问题引入：为什么需要推理优化？

朴素推理虽然能跑通，但有三个根本限制：

- 1. 显存浪费：每个请求独立分配显存，导致碎片化
- 2. 计算浪费：早完成的请求等待最慢的请求，导致 GPU 空转
- 3. 延迟过高：没有批处理，每个请求单独计算

vLLM 通过连续批处理 + PagedAttention来弥补：

朴素推理: "每个请求独立处理，显存碎片化，计算浪费"
vLLM: "连续批处理，显存分页，高吞吐低延迟"

- > 类比：朴素推理像是每个顾客单独结账，vLLM 像是超市收银台。
- > 收银台可以同时处理多个顾客，效率更高。

数学推导：TTFT/TPOT/吞吐

问题设定：

- TTFT（Time to First Token）：首 token 延迟
- TPOT（Time Per Output Token）：每 token 生成时间
- 吞吐（Throughput）：每秒生成的 token 数

推导过程：

Step 1: TTFT 测量

$$TTFT = t_{\text{first_token}} - t_{\text{request_start}}$$

测量方法：

- 记录请求开始时间 $t_{\text{request_start}}$
- 记录第一个 token 生成时间 $t_{\text{first_token}}$
- $TTFT = t_{\text{first_token}} - t_{\text{request_start}}$

Step 2: TPOT 测量

$$TPOT = (t_{\text{last_token}} - t_{\text{first_token}}) / (n_{\text{tokens}} - 1)$$

- 测量方法：
- 记录第一个 token 生成时间 `t_first_token`
 - 记录最后一个 token 生成时间 `t_last_token`
 - $TPOT = (t_last_token - t_first_token) / (n_tokens - 1)$

Step 3: 吞吐测量

$Throughput = total_tokens / total_time$

- 测量方法：
- 统计所有请求生成的 token 总数 `total_tokens`
 - 统计总耗时 `total_time`
 - $Throughput = total_tokens / total_time$

- 关键洞察：
- TTFT 主要由 prefill 阶段决定（处理输入 token）
 - TPOT 主要由 decode 阶段决定（生成输出 token）
 - 吞吐取决于批处理大小和 GPU 利用率

历史脉络：推理优化演进

- 2018: 朴素推理（逐请求处理）
- ↓ 显存浪费，计算浪费
- 2020: 静态批处理（Static Batching）
- ↓ 早完成的请求等待最慢的请求
- 2022: 连续批处理（Continuous Batching, Orca）
- ↓ 动态添加/移除请求
- 2023: PagedAttention（vLLM）
- ↓ 显存分页，减少碎片
- 2024: 投机解码（Speculative Decoding）
- ↓ 用小模型加速大模型推理

- 关键论文：
- vLLM / PagedAttention（同一篇，arXiv 2309.06180）：[Efficient Memory Management for Large Language Model Serving with PagedAttention](https://arxiv.org/abs/2309.06180)
 - Orca: [Orca: A Distributed Serving System for Transformer-Based Generative Models](https://www.usenix.org/conference/osdi22/presentation/yu)

环境与版本策略 (⚠ 全课程最重要的安装决策)

	适合	代价
M latest** (vllm 与其 torch 依赖版本以 pip 实际解析为准，装完用 `python -c "import vllm; print(vllm.__version__)"` 确认)	教学时效最好	~5GB 下载，与
好 pin torch==2.5.1+cu121)	不想再建 venv	已知 flash-attn 角

```
`bash
```

方案 A：

```
uv venv .venv-vllm && source .venv-vllm/bin/activate
```

```
pip install vllm # CUDA 12.x wheel ; 4090(sm89) 完整支持
python -c "import vllm; print(vllm.__version__)"
```

你有什么	能做什么
CPU only	脚本 01 的基线思想可读；vLLM 部分用 Colab（免费 T4）
1×4090	全部内容（0.5B 模型显存无压力；GPTQ/AWQ 4bit 与 n-gram 投机解码都可玩）

学习地图

指标定义 + naive 基线（01：手写侧） ← 点
↓ 同模型同 prompt 同指标
vLLM 离线 → 服务 → benchmark（02） ← 线 → 面
↓ 量化服务 / n-gram 投机解码
三行对比表填空完成 → 面试即用的实证

课后作业

每章末尾有思考题（<details> 折叠答案）。全部学完后：
[Assignment 14](../../assignments/assignment_14/)

相关资源

- [vLLM](https://github.com/vllm-project/vllm) (docs.vllm.ai + examples/ 树是最好的教程)
 - PagedAttention (arXiv 2309.06180) · Orca (OSDI'22)
 - [SGLang](https://github.com/sgl-project/sglang) (32.9k, 对照引擎) · [llama.cpp](https://github.com/ggml-org/llama.cpp) (端侧/GGUF)
- [← 上一章：Part 13 数据工程](../../Part13_data_engineering/tutorial/README.md) | [下一章：Part 15 多模态理解 →](../../Part15_vision_language/tutorial/README.md)

01_naive_baseline

01 — 朴素基线与对比设计：TTFT/TPOT/吞吐怎么测

- > "vLLM 快 20 倍"这种话在面试里没有价值；**同模型、同 prompt、同指标下快 X 倍，
- > 数据在这** 才有价值。本章先建立测量方法（跑
- > [scripts/01_naive_generate_baseline.py](../scripts/01_naive_generate_baseline.py)），
- > 产出一张留空的对比表——02 章 vLLM 来填空。

学习目标

完成本章后，你将能够：

- 手写 TTFT/TPOT/吞吐的测量代码
- 解释 每个指标的含义和测量陷阱
- 设计一个公平的 serving 对比实验
- 识别 异步陷阱、padding 陷阱等常见错误

前置知识

必须掌握：

- Part 8 06 章：TTFT/TPOT/goodput 定义、memory-bound 直觉
- Part 9 02 章：内存墙（为什么批处理能赢）

理论背景

问题引入：为什么需要测量指标？

推理部署虽然能跑通，但需要量化指标来评估性能：

1. TTFT（首 token 延迟）：用户"反应快不快"
2. TPOT（每 token 生成时间）：用户"打字机流畅度"
3. 吞吐（Throughput）：服务方"能同时处理多少请求"

> 类比：TTFT 像是餐厅上第一道菜的速度，TPOT 像是后续上菜的速度，
> 吞吐像是餐厅同时能服务多少桌客人。

数学推导：指标测量方法

问题设定：

- TTFT：首 token 延迟
- TPOT：每 token 生成时间
- 吞吐：每秒生成的 token 数

推导过程：

Step 1: TTFT 测量

$TTFT = t_{\text{first_token}} - t_{\text{request_start}}$

测量方法：

- 记录请求开始时间 $t_{\text{request_start}}$
- 记录第一个 token 生成时间 $t_{\text{first_token}}$
- $TTFT = t_{\text{first_token}} - t_{\text{request_start}}$

Step 2: TPOT 测量

$TPOT = (t_{\text{last_token}} - t_{\text{first_token}}) / (n_{\text{tokens}} - 1)$

测量方法：

- 记录第一个 token 生成时间 $t_{\text{first_token}}$

- 记录最后一个 token 生成时间 $t_{\text{last_token}}$
- $\text{TPOT} = (t_{\text{last_token}} - t_{\text{first_token}}) / (n_{\text{tokens}} - 1)$

Step 3: 吞吐测量

$\text{Throughput} = \text{total_tokens} / \text{total_time}$

测量方法：

- 统计所有请求生成的 token 总数 total_tokens
- 统计总耗时 total_time
- $\text{Throughput} = \text{total_tokens} / \text{total_time}$

关键洞察：

- TTFT 主要由 prefill 阶段决定（处理输入 token）
- TPOT 主要由 decode 阶段决定（生成输出 token）
- 吞吐取决于批处理大小和 GPU 利用率

代码实现

1. 指标的可操作定义（测量代码里怎么算）

运行 [scripts/01_naive_generate_baseline.py](../scripts/01_naive_generate_baseline.py) 验证以下代码。

TTFT（首 token 延迟）：提交请求 → 第一个 token 到达。prefill 主导，用户"反应快不快"

TPOT（每 token 间隔）： $(\text{总时间} - \text{TTFT}) / (n_{\text{tokens}} - 1)$ 。decode 主导，"打字机流畅度"

吞吐 throughput：全体请求 tok/s 合计（服务方视角）

p50/p90：分布式 serving 的真实体验由尾部决定，别只报平均

E2E 延迟 $\approx \text{TTFT} + \text{TPOT} \times (\text{输出 token 数} - 1)$

形状追踪：测量过程

```
| TTFT/TPOT 测量过程 |
| |
| 输入: prompt (str) |
| ↓ tokenize |
| input_ids: (1, seq_len) |
| ↓ prefill (处理输入) |
| 第一个 token 生成 ← 记录 t_first_token |
| ↓ decode (逐个生成) |
| token 2, 3, ... ← 逐个记录时间 |
| ↓ 最后一个 token |
| t_last_token ← 记录 |
| |
| 计算: |
| TTFT = t_first_token - t_request_start |
```

| TPOT = (t_last_token - t_first_token) / (n_tokens - 1) |
| 吞吐 = total_tokens / total_time |

△ 测量陷阱（本脚本全部真实处理过——计时点前后共 7 处 `torch.cuda.synchronize()`）：
• 异步陷阱：GPU 是异步的——不 `synchronize`/同步读结果，测到的是"提交 kernel"而非"算完"的时间（Part 9 01 章）。本脚本的 TTFT 单步探测、逐请求循环、静态批对照三处计时都在掐表前后显式同步，保证测的是完成时刻（新版 transformers 的 `generate` 内部已带若干同步点，但依赖内部实现是脆弱的——自己 `sync` 才是契约）；
• padding 陷阱：decoder-only 批处理必须左 padding（右 padding 会把位置算歪）；
• 首 token 单独测：HF `generate` 是一次性调用，TTFT 需要用 `max_new_tokens=1` 的独立探测来近似——工程上 serving 引擎会流式返回，天然可测。

2. 基线结果（实测）

> 环境：RTX 4090, torch 2.6.0+cu124, transformers 4.57.6,
> Qwen2.5-0.5B-Instruct, 64 请求 × 32 token, greedy, 全部计时点显式同步。
> （脚本头两行会打印你自己的环境；数字随机器状态略有波动属正常——方向不变。
> 本页早期版本引用过 181 tok/s 等未受控数字，已全部替换为复跑实测值；
> 吞吐口径修正：分母只含 64 次正式 `generate` 的计时段合计，不含 TTFT 单步探测
> ——早期版本把探测时间计入总时长却不计其 token，吞吐被系统性低估约 5-10%。）

[1] 逐请求循环（serving 反模式）：

TTFT p50/p90 : 7.5 / 7.6 ms

TPOT p50/p90 : 6.2 / 6.3 ms

吞吐 : 158 tok/s（计时段 12.95s；wall 13.44s 含 TTFT 探测，不作分母）

[2] 静态批处理（batch=8）: 1071 tok/s（吞吐 × 6.8 !）

——但早完成的请求陪跑到最慢的：这就是 Orca 论文要杀死的"static batching 浪费"

• 静态批处理已经赢近 7 倍（6.8×）：权重只读一次喂 8 个请求（memory-bound 的直接推论）。
vLLM 的增量 = 连续批处理（早走早换人）+ PagedAttention（batch 开得更大）+ prefix caching。

3. 对比实验设计（02 章的填空表）

指标	naive 循环（本脚本）	vLLM（02 章）	差异来自
吞吐 tok/s	158（实测）	?	连续批处理
TTFT p50	7.5 ms（实测）	?	prefill 调度/chunked prefill
TPOT p50	6.2 ms（实测）	?	decode batch 更大 + CUDA graphs
KV 显存	每请求整块预留	?	PagedAttention

> 公平性三原则：同模型同 dtype、同 prompt 集（脚本内置的固定 64 条）、同 `max_new_tokens`。
> 换任何一个，数字就不可比。

工程实践

调试展示：常见错误与修复

错误 1：异步陷阱

症状：

```
`python
start = time.time()
output = model.generate(input_ids, max_new_tokens=32)
end = time.time()
print(f"耗时: {end - start:.3f}s") # 输出: 耗时: 0.001s (错误！)
```

原因：GPU 是异步的，`generate` 立即返回，不等待计算完成

解法：

```
`python
start = time.time()
output = model.generate(input_ids, max_new_tokens=32)
torch.cuda.synchronize() # 等待 GPU 计算完成
end = time.time()
print(f"耗时: {end - start:.3f}s") # 输出: 耗时: 0.123s (正确)
```

错误 2：padding 陷阱

症状：

```
`python
```

右 padding (错误)

```
input_ids = [[1, 2, 3, 0, 0], [4, 5, 0, 0, 0]]
```

左 padding (正确)

```
input_ids = [[0, 0, 1, 2, 3], [0, 0, 0, 4, 5]]
```

原因：decoder-only 模型用因果注意力，右 padding 会把位置算歪

解法：始终使用左 padding

错误 3：TTFT 测量不准

症状：TTFT 测量结果与预期不符

原因：用 `max_new_tokens=32` 测量，包含了后续 token 的生成时间

解法：用 `max_new_tokens=1` 单独测量 TTFT

性能数据 (实测参考)

方法	吞吐 tok/s	TTFT p50	TPOT p50	说明
逐请求循环	158	7.5ms	6.2ms	serving 反模式（实测）
静态批处理 (batch=8)	1071	—（未单独测）	—（未单独测）	权重只读一次（实测）
vLLM (batch=64)	~3000+	~3ms	~2ms	连续批处理 + PagedAttention（预期，未本机实测）

- > 数据来源：本课开发机实测（RTX 4090, torch 2.6.0+cu124, transformers 4.57.6,
- > Qwen2.5-0.5B, 64 请求 × 32 token）；吞吐分母 = 64 次正式 generate 的计时段合计
- > （不含 TTFT 探测/tokenize 开销）；vLLM 行为量级预期，02 章自己跑出来回填。

常见陷阱

陷阱 1：测量环境不一致

- 症状：不同次测量结果差异大
- 原因：GPU 频率、后台进程等干扰
- 解法：测量前清理后台进程，多次测量取平均

陷阱 2：prompt 集不一致

- 症状：不同 prompt 集的结果不可比
- 原因：prompt 长度、复杂度不同
- 解法：使用固定的 prompt 集

陷阱 3：模型配置不一致

- 症状：不同配置的结果不可比
- 原因：dtype、量化方式等不同
- 解法：使用相同的模型配置

最佳实践

测量流程

1. 清理环境：关闭后台进程，确保 GPU 空闲
2. 预热：先跑几个请求预热 GPU
3. 多次测量：至少测量 3 次，取平均
4. 记录配置：记录模型、prompt、batch size 等配置

公平对比原则

1. 同模型：相同的模型权重
2. 同 dtype：相同的精度 (fp16/bf16)
3. 同 prompt：相同的输入数据
4. 同 max_new_tokens：相同的输出长度

学完本部分你能...

- 用代码正确测出 TTFT/TPOT/吞吐的 p50/p90，避开三个测量陷阱
- 解释静态批处理为什么已经赢近 7 倍 ($6.8\times$)、vLLM 又赢在哪
- 设计一个公平的 serving 对比实验
- 识别异步陷阱、padding 陷阱等常见错误

概念检验

<details>

<summary>Q1: 为什么 TTFT 的 p90 和 p50 几乎一样 (7.6 vs 7.5) ? 什么场景下会拉开? </summary>

A: 本基线是串行逐请求——每个请求独占 GPU、无排队, TTFT $\approx$ 恒定的 prefill 时间。

真 serving 在高负载下 TTFT 尾部会被排队+批内干扰拉长 (这正是 P99 SLO 和 goodput 存在的原因)。所以本脚本的 TTFT 是"空载 TTFT", 跟 vLLM 对比时注意负载条件要一致。

</details>

<details>

<summary>Q2: 静态批处理 8 路吞吐 1071 tok/s, 是不是继续加 batch 就线性涨? </summary>

A: 在 memory-bound 区间近似线性 (权重搬运被摊薄), 直到 compute-bound 或显存 (KV) 耗尽——4090 上 0.5B 模型 KV 很小, 瓶颈先出现在调度/内存拷贝。真实大模型上 KV 显存先爆, 这正是 PagedAttention 的用武之地 (02 章日志里 GPU KV cache usage 会印证)。

</details>

<details>

<summary>Q3: 为什么 decoder-only 批处理必须左 padding? </summary>

A: decoder-only 模型用因果注意力, 每个 token 只能看到前面的 token。

右 padding 会把 padding token 放在后面, 导致模型"看到" padding token, 从而影响生成结果。左 padding 把 padding token 放在前面, 模型不会"看到"它们。

</details>

动手实践

<details>

<summary>练习 1: 实现 TTFT 测量函数</summary>

任务: 实现一个函数, 测量首 token 延迟。

验收标准:

- [] 输入: 模型、input_ids、max_new_tokens
- [] 输出: TTFT (秒)
- [] 正确处理异步陷阱

步骤提示:

`python

```
def measure_ttft(model, input_ids, max_new_tokens=1):  
    """
```

Steps:

1. 记录开始时间
 2. 调用 model.generate(max_new_tokens=1)
 3. 同步 GPU
 4. 记录结束时间
 5. 返回 TTFT
- ```
 """
```

## TODO: Implement

pass

</details>

<details>

<summary>练习 2: 实现吞吐测量函数</summary>

任务：实现一个函数，测量批量请求的吞吐。

验收标准：

- [] 输入：模型、prompts、max\_new\_tokens
- [] 输出：吞吐 (tok/s)
- [] 正确统计总 token 数

步骤提示：

```
`python
def measure_throughput(model, prompts, max_new_tokens=32):
 """
```

Steps:

1. 记录开始时间
2. 批量调用 model.generate
3. 同步 GPU
4. 记录结束时间
5. 统计总 token 数
6. 计算吞吐

```
"""
```

## TODO: Implement

pass

</details>

<details>

<summary>练习 3: 设计公平对比实验</summary>

任务：设计一个公平的 naive vs vLLM 对比实验。

验收标准：

- [] 列出需要控制的变量
- [] 设计测量流程
- [] 设计结果展示方式

步骤提示：

```
`python
def design_experiment():
 """
```

Steps:

1. 列出需要控制的变量（模型、dtype、prompt、max\_new\_tokens）
2. 设计测量流程（预热、多次测量、取平均）
3. 设计结果展示方式（表格、图表）

```
"""
```

# TODO: Implement

pass

</details>

## 课后作业

完成本章后，去 Assignment 14 完成练习：

[Assignment 14](../../assignments/assignment\_14/)

## 下一步

装 vLLM（两案选一），把填空表填上。

[02 — vLLM 实战与对比](02\_vllm\_serving.md)

## 02\_vllm\_serving

### 02 — vLLM 实战：离线推理 → 服务 → benchmark → 量化/投机解码

> 填空时间。装 vLLM（README 两案），同一模型（Qwen2.5-0.5B-Instruct）、同一批 prompt，> 把 01 章的对比表填完。每一步都标注"对应 Part 8 06 章手写的哪一块"。

## 学习目标

完成本章后，你将能够：

- 完成 vLLM 安装（两案）、离线推理、OpenAI 服务部署
- 用官方 benchmark 出 TTFT/TPOT 分位数并填完对比表
- 部署量化模型与 n-gram 投机解码，解释各自的适用条件
- 把课程的手写模块逐一对应到工业实现，形成"懂原理 + 会工具"的完整叙事

## 前置知识

必须掌握：

- 01 章：指标定义与 naive 基线（158 tok/s / 7.5ms / 6.2ms，待超越）
- Part 8 06 章：PagedAttention/连续批处理/投机解码的手写模拟

## 理论背景

## 问题引入：为什么需要 vLLM？

朴素推理虽然能跑通，但有三个根本限制：

1. 显存浪费：每个请求独立分配显存，导致碎片化
2. 计算浪费：早完成的请求等待最慢的请求，导致 GPU 空转
3. 延迟过高：没有批处理，每个请求单独计算

vLLM 通过连续批处理 + PagedAttention来弥补：

朴素推理: "每个请求独立处理，显存碎片化，计算浪费"

vLLM: "连续批处理，显存分页，高吞吐低延迟"

> 类比：朴素推理像是每个顾客单独结账，vLLM 像是超市收银台。

> 收银台可以同时处理多个顾客，效率更高。

## 数学推导：PagedAttention 的显存优化

问题设定：

- 朴素推理：每个请求独立分配 KV cache 显存
- PagedAttention：KV cache 分页管理

推导过程：

Step 1: 朴素推理的显存浪费

每个请求分配  $\text{max\_seq\_len}$  的 KV cache

实际使用  $\text{only\_used\_seq\_len}$

浪费 =  $\text{max\_seq\_len} - \text{used\_seq\_len}$

示例： $\text{max\_seq\_len}=2048, \text{used\_seq\_len}=512$

浪费 =  $2048 - 512 = 1536$  tokens

浪费率 =  $1536 / 2048 = 75\%$

Step 2: PagedAttention 的分页管理

把 KV cache 分成固定大小的 block

按需分配 block，而非预分配  $\text{max\_seq\_len}$

浪费 = 最后一个 block 的内部碎片

示例： $\text{block\_size}=16, \text{used\_seq\_len}=512$

需要 block 数 =  $\text{ceil}(512 / 16) = 32$

浪费 =  $16 - (512 \% 16) = 16 - 0 = 0$  tokens

浪费率  $\approx 0\%$

关键洞察：

- PagedAttention 把显存碎片化问题转化为分页问题
- $\text{block\_size}$  越小，碎片越少，但管理开销越大
- 实践中  $\text{block\_size}=16$  是常见配置

## 代码实现

## 1. 离线推理（5 行）

```
`python
from vllm import LLM, SamplingParams
llm = LLM(model="Qwen/Qwen2.5-0.5B-Instruct") # 启动日志找 'GPU KV cache size'
outs = llm.generate(["The quick brown fox", "Deep learning models are"],
SamplingParams(temperature=0.7, max_tokens=64))
print(outs[0].outputs[0].text)
```

启动日志三行必看：**# GPU blocks**（KV 池的块数——PagedAttention 的页表容量）、**max seq len**、**GPU KV cache usage**（运行时占用率）。对照手写：Part 8 06 章模拟里“整块预留浪费 41%”的问题，这里因为分页只剩 ~5%（论文口径 <4%）。

## 2. OpenAI 兼容服务

```
`bash
vllm serve Qwen/Qwen2.5-0.5B-Instruct --max-model-len 2048
curl http://localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
-d '{"model": "Qwen/Qwen2.5-0.5B-Instruct", "messages": [{"role": "user", "content": "你好"}]}'
```

概念映射：**--max-model-len** = KV 池上限；连续批处理自动开启（无需配置）；  
**--gpu-memory-utilization** = KV 池占比调参；**--enable-prefix-caching**（新版默认开） = 共享前缀免重算（Part 8 06 章“prefix sharing”）。

## 3. Benchmark（官方脚本，01 章基线的同款指标）

```
`bash
```

服务端起好后：

```
python benchmarks/benchmark_serving.py --backend vllm \
--model Qwen/Qwen2.5-0.5B-Instruct --dataset-name random --num-prompts 64 \
--request-rate inf # 吞吐、TTFT/TPOT 的 p50/p99 一次出齐
```

填空表参考形态（4090 实测量级，你的数字会不同——这正是要自己跑的原因）：  
吞吐从 naive 的 ~158 tok/s 到 数千 tok/s（0.5B 小模型上  $10\times+$ ；7B 上同样量级收益），  
TPOT 反而可能略升（batch 大了 decode 稍慢）但吞吐大涨——serving 的本质是吞吐换延迟。

## 4. 量化服务（Part 8 06 章“手写量化”的工业对应）

```
`bash
```

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct-GPTQ-Int4 # GPTQ/AWQ/FP8 权重直接加载
```

对比：模型文件体积（fp16 ~1GB  $\rightarrow$  int4 ~0.4GB）、KV 不变、吞吐与精度的实测变化

5. n-gram 投机解码（无需 draft 模型——24GB 卡友好）

```
`python
from vllm import LLM, SamplingParams
llm = LLM(model="Qwen/Qwen2.5-0.5B-Instruct",
speculative_config={"method": "ngram", "prompt_lookup_num_tokens": 4})
```

ngram 投机 = prompt lookup：从 prompt 已有文本里检索匹配片段当草稿 token,

所以不需要 draft 模型；prompt\_lookup\_num\_tokens = 每步草稿长度

对比开/关投机解码的吞吐；Part 8 06 章（脚本 09）的"接受率  $\alpha$ "概念 = vLLM 日志里的 acceptance rate

6. 总账：本课程"手写 → 工具"的完整对照

| 手写（Part 8/9）                            | vLLM/工业           | 你能做的验证               |
|-----------------------------------------|-------------------|----------------------|
| KV cache 字典（P7）                         | PagedAttention 块表 | 日志 KV usage + 显存对比   |
| 分页模拟（P8 06 章脚本 09：41%→5%）               | 真实 <4%            | 同 batch 下 KV 显存      |
| 手写量化（P8 06 章）                           | GPTQ/AWQ 权重       | 文件体积 + ppl/acc       |
| 手写投机解码（P8 06 章， $\alpha \approx 0.60$ ） | n-gram/EAGLE      | acceptance rate + 吞吐 |
| naive/静态批基线（P14 01）                     | 连续批处理             | 三行对比表                |

- > 面试结论模板："我在 4090 上用 Qwen2.5-0.5B 做过 naive→vLLM 的对比，吞吐 158→N tok/s,
- > 差异归因于连续批处理和 PagedAttention——我的手写模拟复现了同样的方向性。"—
- > 这段话的每个数字你都能现场推导。

工程实践

调试展示：常见错误与修复

错误 1：vLLM 安装失败

症状：

```
`ERROR: Could not find a version that satisfies the requirement vllm
```

原因：Python 版本不对，或 CUDA 版本不兼容

解法：

```
`bash
```

## 检查 Python 版本

```
python --version # 需要 3.8+
```

## 检查 CUDA 版本

```
nvcc --version # 需要 11.8+
```

## 使用正确的版本

```
pip install vllm --extra-index-url https://download.pytorch.org/whl/cu118
```

### 错误 2：显存不足

症状：

```
CUDA out of memory. Tried to allocate 2.00 MiB
```

原因：模型太大，或 KV cache 太大

解法：

```
`bash
```

## 减小 max-model-len

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct --max-model-len 1024
```

## 或减小 gpu-memory-utilization

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct --gpu-memory-utilization 0.8
```

### 错误 3：服务启动失败

症状：

```
Error: Address already in use
```

原因：端口被占用

解法：

```
`bash
```

## 查找占用端口的进程

```
lsof -i:8000
```

## 杀掉进程



```
kill -9 <PID>
```

或使用其他端口

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct --port 8001
```

性能数据（实测 + 预期标注）

| 方法              | 吞吐 tok/s | TTFT p50 | TPOT p50 | 显存占用   | 口径         |
|-----------------|----------|----------|----------|--------|------------|
| 逐请求循环           | 158      | 7.5ms    | 6.2ms    | ~2GB   | 实测         |
| 静态批处理 (batch=8) | 1071     | —        | —        | ~4GB   | 实测（01 章脚本） |
| vLLM (batch=64) | ~3000+   | ~3ms     | ~2ms     | ~3GB   | ⚠ 预期，未本机实测 |
| vLLM + 量化       | ~4000+   | ~2ms     | ~1.5ms   | ~1.5GB | ⚠ 预期，未本机实测 |

- > 实测口径：本课开发机（RTX 4090，torch 2.6.0+cu124，transformers 4.57.6，
- > Qwen2.5-0.5B，64 请求 × 32 token，含 `torch.cuda.synchronize()` 的真实计时）。
- > ⚠ 两行 vLLM 数字是同量级预期（来自官方论文/社区 benchmark 的量级），
- > 本课开发环境未安装 vLLM，未做同机实测——02 章实操跑完后请用你的数字覆盖，
- > 与 01 章表格的"(预期)"标注同一约定：没实测的一律标出来。

常见陷阱

陷阱 1：版本不兼容

- 症状：安装失败，或运行时报错
- 原因：Python/CUDA/PyTorch 版本不兼容
- 解法：使用官方推荐的版本组合

陷阱 2：显存估算不准

- 症状：服务启动后 OOM
- 原因：没有考虑 KV cache 的显存开销
- 解法：使用 vLLM 的显存估算功能，或手动计算

陷阱 3：量化模型精度下降

- 症状：量化后模型效果变差
- 原因：量化方式不合适，或量化参数不对
- 解法：尝试不同的量化方式（GPTQ/AWQ/FP8），调整量化参数

最佳实践

配置推荐

| 参数                       | 推荐值       | 说明      |
|--------------------------|-----------|---------|
| `max-model-len`          | 2048-4096 | 根据任务调整  |
| `gpu-memory-utilization` | 0.9       | 显存利用率   |
| `enable-prefix-caching`  | true      | 共享前缀免重算 |
| `quantization`           | awq/gptq  | 量化方式    |
| `speculative_config`     | ngram     | 投机解码    |

调试流程

- 1. 先用小模型：0.5B 模型快速验证
- 2. 检查日志：查看 KV cache 使用率、batch size 等
- 3. 逐步增大：从 0.5B 到 7B，从 batch 1 到 batch 64
- 4. 监控显存：使用 nvidia-smi 监控显存使用

学完本部分你能...

- 独立完成 vLLM 安装（两案）、离线推理、OpenAI 服务部署
- 用官方 benchmark 出 TTFT/TPOT 分位数并填完对比表
- 部署量化模型与 n-gram 投机解码，解释各自的适用条件
- 把课程的手写模块逐一对应到工业实现，形成"懂原理 + 会工具"的完整叙事

概念检验

<details>

<summary>Q1: vLLM 的 TPOT 有时比 naive 略高，为什么还用它？</summary>

A: 大 batch 下 decode 步变慢（每步算更多请求），但吞吐（tok/s 合计）大增——serving 优化的是"每瓦特/每卡的 token 成本"。单请求延迟敏感场景用小 batch/SLO 路由，吞吐场景用大 batch——goodput 的含义（Part 8 06 章）。

</details>

<details>

<summary>Q2: prefix caching 什么时候收益最大？什么时候没收益？</summary>

A: 共享前缀长且重复率高（系统提示词、few-shot 模板、多轮对话历史）时收益巨大（TTFT 降几倍）；prompt 完全随机时纯开销（查表成本）。看业务 prompt 分布决定开关。

</details>

<details>

<summary>Q3: 量化模型和原模型的精度差距有多大？</summary>

A: 取决于量化方式和模型大小：

- 4bit GPTQ/AWQ：精度下降 1-3%，显存节省 75%
- 8bit FP8：精度下降 <1%，显存节省 50%
- 小模型（<1B）量化后精度下降更明显

</details>

动手实践

<details>

<summary>练习 1: 部署 vLLM 服务</summary>

任务：部署一个 vLLM 服务并测试。

验收标准：

- ☐ 成功安装 vLLM
- ☐ 成功启动服务
- ☐ 成功发送请求并获取响应

步骤提示：

```
`bash
```

## 1. 安装 vLLM

```
pip install vllm
```

## 2. 启动服务

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct --max-model-len 2048
```

## 3. 发送请求

```
curl http://localhost:8000/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{"model": "Qwen/Qwen2.5-0.5B-Instruct", "messages": [{"role": "user", "content": "你好"}]}'
```

</details>

<details>

<summary>练习 2: 运行 benchmark</summary>

任务：运行 vLLM 官方 benchmark 并记录结果。

验收标准：

- ☐ 成功运行 benchmark
- ☐ 记录 TTFT/TPOT/吞吐
- ☐ 与 01 章基线对比

步骤提示：

```
`bash
```

## 运行 benchmark

```
python benchmarks/benchmark_serving.py --backend vllm \
--model Qwen/Qwen2.5-0.5B-Instruct --dataset-name random --num-prompts 64 \
--request-rate inf
```

## 记录结果

吞吐: ??? tok/s

TTFT p50: ??? ms

TPOT p50: ??? ms

</details>

<details>

<summary>练习 3: 测试量化模型</summary>

任务：部署量化模型并对比精度和性能。

验收标准：

- ☐ 成功部署量化模型
- ☐ 记录显存占用
- ☐ 记录吞吐变化
- ☐ 测试精度变化

步骤提示：

`bash

## 部署量化模型

vllm serve Qwen/Qwen2.5-0.5B-Instruct-GPTQ-Int4

## 记录显存占用

nvidia-smi

## 记录吞吐变化

```
python benchmarks/benchmark_serving.py --backend vllm \
--model Qwen/Qwen2.5-0.5B-Instruct-GPTQ-Int4 --dataset-name random --num-prompts 64
```

</details>

## 课后作业

完成本章后，去 Assignment 14 完成练习：

[Assignment 14](../../assignments/assignment\_14/)

## 课程毕业

Part 1-14 完整覆盖：从手写 bigram 到工业 RL 与 serving。

回 [课程总览](../../README.md) 规划你的 Part 15（多模态）与面试（docs/llm\_interview\_guide.md）。